

AI ASSISTED CODING

ASSIGNMENT-8

NAME: Varshitha. G

HTNO: 2303A51103

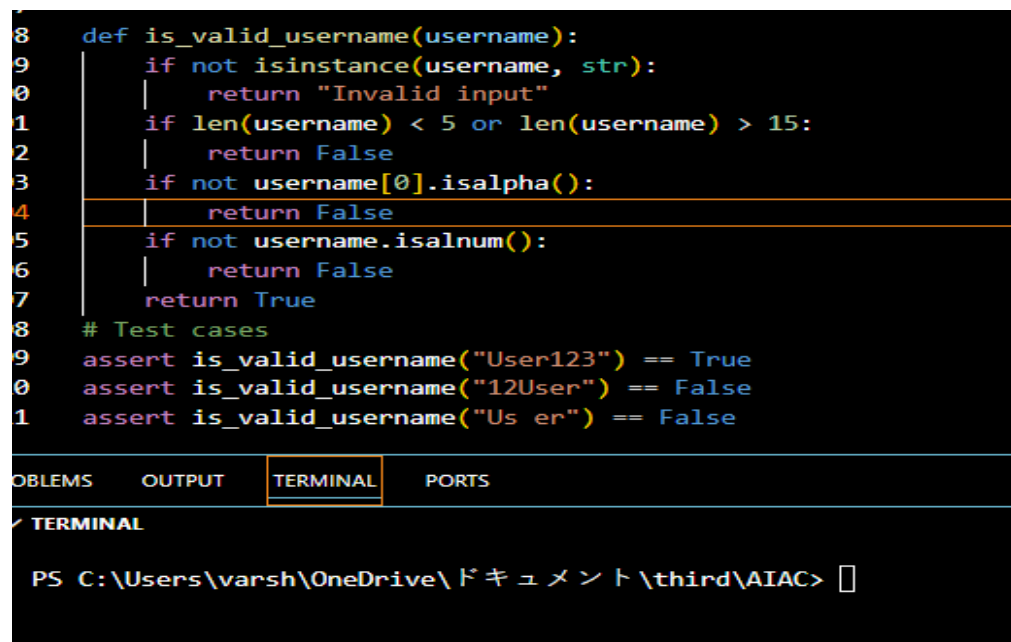
Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username` (username) and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

```
8 def is_valid_username(username):
9     if not isinstance(username, str):
10         return "Invalid input"
11     if len(username) < 5 or len(username) > 15:
12         return False
13     if not username[0].isalpha():
14         return False
15     if not username.isalnum():
16         return False
17     return True
18 # Test cases
19 assert is_valid_username("User123") == True
20 assert is_valid_username("12User") == False
21 assert is_valid_username("Us er") == False
```



PROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> □

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

```
113 def classify_value(x):
114     if x==0:
115         return "Zero"
116     if isinstance(x, (int, float)):
117         if x%2==0:
118             return "Even"
119         else:
120             return "Odd"
121     return "Invalid Input"
122 # Test cases
123 assert classify_value(8) == "Even"
124 assert classify_value(7) == "Odd"
125 assert classify_value("abc") == "Invalid Input"
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```
● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh\AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") == True
```

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

```
126
127 def is_palindrome(s):
128     if not isinstance(s, str):
129         return "Invalid input"
130     s = s.replace(" ", "").lower()
131     return s == s[::-1]
132 # Test cases
133 assert is_palindrome("Madam") == True
134 assert is_palindrome("A man a plan a canal Panama") == True
135 assert is_palindrome("Python") == False
136
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
> ▼ TERMINAL PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

- o Must contain @ and .
- o Must not start or end with special characters.
- o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```
37 def validate_email(email):
38     if not isinstance(email, str):
39         return "Invalid input"
40     if "@" not in email or "." not in email:
41         return False
42     if email.count("@") != 1:
43         return False
44     if email.startswith("@") or email.endswith("@"):
45         return False
46     if email.startswith(".") or email.endswith("."):
47         return False
48     return True
49
50 assert validate_email("user@example.com") == True
51 assert validate_email("userexample.com") == False
52 assert validate_email("@gmail.com") == False
```

PROBLEMS OUTPUT **TERMINAL** PORTS ...

✓ **TERMINAL**

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/OneDrive\ドキュメント\third\AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> 
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).
- Test Cases to Design:
 - o Normal case: 6 → True, 10 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Larger case: 28.
- Requirement: Validate correctness with assertions.

```
154 def is_perfect_number(n):
155     if not isinstance(n, int) or isinstance(n, bool):
156         return "Invalid input"
157     if n < 1:
158         return "Invalid input"
159     sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
160     return sum_of_divisors == n
161 assert is_perfect_number(6) == True
162 assert is_perfect_number(10) == False
163 assert is_perfect_number(1) == False
164 assert is_perfect_number(-5) == "Invalid input"
165 assert is_perfect_number(0) == "Invalid input"
166 assert is_perfect_number(28) == True
```

PROBLEMS OUTPUT **TERMINAL** PORTS ...

> ▼ TERMINAL

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
 - o Normal case: 12 → True, 15 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 945.

Requirement: Validate correctness with unittest

```

167
168 # write a function to check abundant number and validate the implementation using unittest
169 import unittest
170 def is_abundant_number(n):
171     if not isinstance(n, int) or isinstance(n, bool):
172         return "Invalid input"
173     if n < 1:
174         return "Invalid input"
175     sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
176     return sum_of_divisors > n
177 class TestIsAbundantNumber(unittest.TestCase):
178     def test_abundant_numbers(self):
179         self.assertTrue(is_abundant_number(12))
180         self.assertTrue(is_abundant_number(18))
181         self.assertTrue(is_abundant_number(20))
182     def test_non_abundant_numbers(self):
183         self.assertFalse(is_abundant_number(6))
184         self.assertFalse(is_abundant_number(28))
185         self.assertFalse(is_abundant_number(1))
186     def large_cases(self):
187         self.assertTrue(is_abundant_number(945))
188         self.assertFalse(is_abundant_number(496))
189     def test_invalid_input(self):
190         self.assertEqual(is_abundant_number(-5), "Invalid input")
191         self.assertEqual(is_abundant_number(0), "Invalid input")
192         self.assertEqual(is_abundant_number(3.5), "Invalid input")
193         self.assertEqual(is_abundant_number("string"), "Invalid input")
194         self.assertEqual(is_abundant_number(True), "Invalid input")
195 if __name__ == '__main__':
196     unittest.main()
197

```

PROBLEMS OUTPUT **TERMINAL** PORTS

> ▾ TERMINAL



Ran 3 tests in 0.000s

OK

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).
- Test Cases to Design:
 - o Normal case: 8 → True, 12 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 546.

Requirement: Validate correctness with pytest.

```
# write a function to check if a number is deficient (sum of divisors is less than the num
def is_deficient_number(n):
    if not isinstance(n, int) or isinstance(n, bool):
        return "Invalid input"
    if n < 1:
        return "Invalid input"
    sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
    return sum_of_divisors < n
def test_is_deficient_number():
    assert is_deficient_number(8) == True
    assert is_deficient_number(12) == True
    assert is_deficient_number(1) == True
    assert is_deficient_number(546) == False
    assert is_deficient_number(28) == False
    assert is_deficient_number(-5) == "Invalid input"
    assert is_deficient_number(0) == "Invalid input"
    assert is_deficient_number(3.5) == "Invalid input"
    assert is_deficient_number("string") == "Invalid input"
    assert is_deficient_number(True) == "Invalid input"
```

BLEMS OUTPUT **TERMINAL** PORTS ...

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █
```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```
18 # Write a function LeapYearChecker and validate its implementation using 10 pytest test cases
19 def is_leap_year(year):
20     if not isinstance(year, int) or isinstance(year, bool):
21         return "Invalid input"
22     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
23         return True
24     return False
25 def test_is_leap_year():
26     assert is_leap_year(2020) == True
27     assert is_leap_year(1900) == False
28     assert is_leap_year(2000) == True
29     assert is_leap_year(2021) == False
30     assert is_leap_year(2400) == True
31     assert is_leap_year(-4) == "Invalid input"
32     assert is_leap_year(0) == "Invalid input"
33     assert is_leap_year(3.5) == "Invalid input"
34     assert is_leap_year("string") == "Invalid input"
35     assert is_leap_year(True) == "Invalid input"
```

PROBLEMS OUTPUT **TERMINAL** PORTS

✓ TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C ...
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █
```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```
238 #Write a function SumOfDigits and validate its implementation using 7 pytest test cases.
239 def sum_of_digits(n):
240     if not isinstance(n, int) or isinstance(n, bool):
241         return "Invalid input"
242     return sum(int(digit) for digit in str(abs(n)))
243 def test_sum_of_digits():
244     assert sum_of_digits(123) == 6
245     assert sum_of_digits(-456) == 15
246     assert sum_of_digits(0) == 0
247     assert sum_of_digits(9999) == 36
248     assert sum_of_digits(1001) == 2
249     assert sum_of_digits(3.5) == "Invalid input"
250     assert sum_of_digits("string") == "Invalid input"
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

```
.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python39-64/Python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.


```

52 # Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.
53 def bubble_sort(arr):
54     if not isinstance(arr, list) or any(not isinstance(x, (int, float)) for x in arr):
55         return "Invalid input"
56     n = len(arr)
57     for i in range(n):
58         for j in range(0, n-i-1):
59             if arr[j] > arr[j+1]:
60                 arr[j], arr[j+1] = arr[j+1], arr[j]
61     return arr
62
63 def test_bubble_sort():
64     assert bubble_sort([64, 34, 25, 12, 22, 11, 90]) == [11, 12, 22, 25, 34, 64, 90]
65     assert bubble_sort([5, 1, 4, 2, 8]) == [1, 2, 4, 5, 8]
66     assert bubble_sort([]) == []
67     assert bubble_sort([1]) == [1]
68     assert bubble_sort([3.5, 2.1, 4.8]) == [2.1, 3.5, 4.8]
69     assert bubble_sort([-5, -2, -3]) == [-5, -3, -2]
70     assert bubble_sort([0]) == [0]
71     assert bubble_sort([1.5]) == [1.5]
72     assert bubble_sort([-1.5]) == [-1.5]
73     assert bubble_sort([3.5]) == [3.5]
74     assert bubble_sort([-3.5]) == [-3.5]
75     assert bubble_sort([0.0]) == [0.0]
76     assert bubble_sort([-0.0]) == [-0.0]
77     assert bubble_sort([1e10, -1e10]) == [-1e10, 1e10]
78     assert bubble_sort([-1e10, -1e10]) == [-1e10, -1e10]
79     assert bubble_sort([1e10, 1e10]) == [1e10, 1e10]
80     assert bubble_sort([3.14, 2.71]) == [2.71, 3.14]
81     assert bubble_sort([-3.14, -2.71]) == [-3.14, -2.71]
82     assert bubble_sort([0.001, -0.001]) == [-0.001, 0.001]
83     assert bubble_sort([-0.001, -0.001]) == [-0.001, -0.001]
84     assert bubble_sort([1000000]) == [1000000]
85     assert bubble_sort([-1000000]) == [-1000000]
86     assert bubble_sort([999999999]) == [999999999]

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe
rsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

```

287 # Write a function ReverseString and validate its implementation using 5 unittest test cases
288 def reverse_string(s):
289     if not isinstance(s, str):
290         return "Invalid input"
291     return s[::-1]
292
293 import unittest
294 class TestReverseString(unittest.TestCase):
295     def test_reverse_string(self):
296         self.assertEqual(reverse_string("hello"), "olleh")
297         self.assertEqual(reverse_string("Python"), "nohtyP")
298         self.assertEqual(reverse_string(""), "")
299         self.assertEqual(reverse_string("A man a plan a canal Panama"), "amanaP lanac a nalp a nam A")
300         self.assertEqual(reverse_string(123), "Invalid input")
301
302 if __name__ == '__main__':
303     unittest.main()

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

rsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-8.py
.
-----
Ran 1 test in 0.000s

OK
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task 12 :

Write a function `AnagramChecker` and validate its implementation using 10 unittest testcases.

```
303 # Write a function AnagramChecker and validate its implementation using 10 unittest test cases.
304 def are_anagrams(str1, str2):
305     if not isinstance(str1, str) or not isinstance(str2, str):
306         return "Invalid input"
307     return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
308 import unittest
309 class TestAnagramChecker(unittest.TestCase):
310     def test_anagrams(self):
311         self.assertTrue(are_anagrams("listen", "silent"))
312         self.assertTrue(are_anagrams("Triangle", "Integral"))
313     def test_non_anagrams(self):
314         self.assertFalse(are_anagrams("hello", "world"))
315         self.assertFalse(are_anagrams("Python", "Java"))
316     def test_invalid_input(self):
317         self.assertEqual(are_anagrams(123, "abc"), "Invalid input")
318         self.assertEqual(are_anagrams("abc", 123), "Invalid input")
319         self.assertEqual(are_anagrams(True, "abc"), "Invalid input")
320         self.assertEqual(are_anagrams("abc", False), "Invalid input")
321     def test_edge_cases(self):
322         self.assertTrue(are_anagrams("", ""))
323         self.assertTrue(are_anagrams("a", "a"))
324     def test_case_sensitivity(self):
325         self.assertTrue(are_anagrams("Listen", "Silent"))
326         self.assertTrue(are_anagrams("Triangle", "Integral"))
327     def test_whitespace_handling(self):
328         self.assertTrue(are_anagrams("Dormitory", "Dirty Room"))
329         self.assertTrue(are_anagrams("Conversation", "Voices Rant On"))
330     def test_special_characters(self):
331         self.assertTrue(are_anagrams("A!B@C#", "C#B@A!"))
332         self.assertFalse(are_anagrams("A!B@C#", "C#B@D!"))
333     def test_numeric_strings(self):
334         self.assertTrue(are_anagrams("123", "321"))
335         self.assertFalse(are_anagrams("123", "322"))
336     def test_mixed_input(self):
337         self.assertEqual(are_anagrams("123", 321), "Invalid input")
338         self.assertEqual(are_anagrams(123, "321"), "Invalid input")
339 if __name__ == '__main__':
340     unittest.main()
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

Ran 9 tests in 0.007s

OK

Task 13 :

Write a function `ArmstrongChecker` and validate its implementation using 8 unittest test cases.

```

341
342 # Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.
343 def is_armstrong_number(n):
344     if not isinstance(n, int) or isinstance(n, bool):
345         return "Invalid input"
346     num_str = str(abs(n))
347     num_digits = len(num_str)
348     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
349     if armstrong_sum == abs(n):
350         return True
351     return False
352 import unittest
353 class TestArmstrongChecker(unittest.TestCase):
354     def test_armstrong_numbers(self):
355         self.assertTrue(is_armstrong_number(153))
356         self.assertTrue(is_armstrong_number(0))
357         self.assertTrue(is_armstrong_number(9474))
358     def test_non_armstrong_numbers(self):
359         self.assertFalse(is_armstrong_number(145))
360         self.assertFalse(is_armstrong_number(10))
361         self.assertFalse(is_armstrong_number(123))
362     def test_invalid_input(self):
363         self.assertEqual(is_armstrong_number(-153), "Invalid input")
364         self.assertEqual(is_armstrong_number(3.5), "Invalid input")
365         self.assertEqual(is_armstrong_number("string"), "Invalid input")
366         self.assertEqual(is_armstrong_number(True), "Invalid input")
367 if __name__ == '__main__':
368     unittest.main()

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python
Ran 3 tests in 0.001s

```

```

FAILED (failures=1)

```

```

❖ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```